

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Final Project Report

Topic 2 – AI Food Analyzer

Team: Team_3_M503 **Date:** Sept. 19, 2026
Repo: https://github.com/atlukhanovmurad2-ui/swe_project **Tag:** v1.0-final
Members: Murad Atlukhanov
 (@atlukhanovmurad2-ui), Zhala Afandiyeva (@zhafandiyeva) Penah Quliyev (@pnah6699-maker)

Executive Summary

Required. Exactly one paragraph (5–7 sentences). What you built, the headline concurrency speedup you measured, your headline robustness story, and the single most surprising thing the team learned. *This is the first thing the grader reads. If you cannot write it clearly, you do not yet understand your own project.*

Your paragraph should answer: What problem did the system solve? What is the one number that proves the concurrency design works? What is the one failure mode you handled well? What would you change if you started over?

We built an AI Food Analyzer that accepts a meal image and returns ingredients, estimated portions, nutrition values, totals, and a macronutrient split. Users can run it through a CLI or a FastAPI service, while PostgreSQL stores analysis history when configured. Six simulated nutrition requests took 1.206 seconds sequentially and 0.208 seconds concurrently, a 5.8-times speedup. The system validates files, retries provider failures, limits parallel work, and returns partial results when one nutrition lookup fails. Offline providers keep tests repeatable without network access. The most useful lesson was that one failed ingredient should not remove successful results for the rest of the meal. With more time, we would add strict request timeouts and complete 429 handling.

Contents

Executive Summary	1
1 Project Overview	2
1.1 Problem framing & scope	2
1.2 Provider choice and the AI module contract	3
2 Architecture	3
2.1 Module map	3
2.2 OOP design & key abstractions	4
2.3 Data flow across module boundaries	5

3	Concurrency & Performance	5
3.1	Why this concurrency model	5
3.2	Sequential-vs-concurrent benchmark	5
3.3	Rate limit & politeness	6
4	Robustness & Error Handling	6
4.1	Wrapping the AI module	6
4.2	Failure-mode analysis (one concrete incident)	7
4.3	Input validation	7
5	Storage & Caching	7
5.1	Schema	7
5.2	Caching policy	8
6	Testing	8
6.1	Strategy & coverage	8
6.2	Notable tests	8
7	Deployment & Reproducibility	9
7.1	Docker image	9
7.2	Configuration	9
8	Limitations & Future Work	10
9	Tools & Acknowledgements	10
	References	10
A	Appendix — Reproducing the benchmark	11

1. Project Overview

1.1 Problem framing & scope

Required. 2–3 paragraphs. State the topic problem in your own words. List the inputs, outputs, and any user-facing entry points (CLI commands, HTTP endpoints). State which provider stack you used (LLM, embedding, USDA, web search).

Beyond the obvious: which parts of the TOPIC.md spec did your team explicitly decide *not* to do, and why? Where did you tighten the spec? Where did you loosen it?

The system solves the problem of estimating nutrition from a cooked meal that has no food label. Its input is a JPEG or PNG meal image. Its output contains identified ingredients, estimated grams, calories, protein, carbohydrates, fat, totals, confidence values, and warnings.

The CLI provides `analyze` and `history`. The HTTP service provides `POST /analyze`, `GET /health`, `GET /history`, and `GET /history/{id}`. The normal provider stack is a configurable vision-language model and USDA FoodData Central. During development, OpenAI with `gpt-4o-mini` was used; offline providers support demos and tests. We did not build user accounts, a web interface, medical advice, or permanent image storage. We tightened the specification by checking file signatures, not only extensions. We also chose to return a partial result when one USDA lookup fails instead of failing the whole request.

1.2 Provider choice and the AI module contract

Required. Name the LLM, embedding, and any auxiliary provider you used, and the model identifiers (`claude-sonnet-4-6`, `text-embedding-3-small`, etc.). State what `ai/` functions you call. *Confirm in one sentence that you did not modify the public interface of the provided `ai/` package.*

Push deeper: how does your code defend against a malformed model response? What happens when the provider returns valid JSON that is semantically wrong (wrong topic enum, negative grams, etc.)?

The tested setup used `LLM_PROVIDER=openai`, `LLM_MODEL=gpt-4o-mini`, and `NUTRITION_PROVIDER=usda`. This topic does not need an embedding or web-search provider. The SE layer calls `ai.identify_ingredients`, `ai.get_nutrition_provider`, and `ai.compute_totals`.

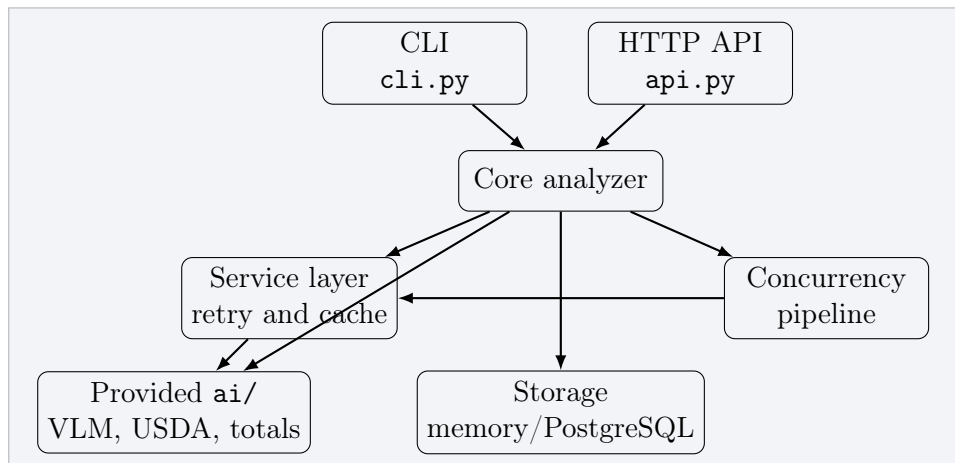
Pydantic validates provider output before the main pipeline uses it. The provided ingredient schema rejects empty names, non-positive or extremely large gram estimates, and confidence outside 0–1. SE-layer models forbid unexpected fields. An empty list becomes `unknown_meal`. We did not modify the public interface of the provided `ai/` package.

2. Architecture

2.1 Module map

Required. An architecture diagram (boxes-and-arrows or sequence diagram) showing the main modules and their dependencies. Identify on the diagram: the boundary of the provided `ai/` package, your service layer that wraps it, your core business logic, your concurrency layer, your storage layer, and your CLI/HTTP entry points.

Interpret your own diagram: which arrows cross the AI-module boundary? Which arrows cross the storage boundary? If you had to swap providers (Anthropic → OpenAI), how many files would change?



Module-by-module summary (1–2 sentences each):

The CLI parses commands, selects offline providers when requested, and prints a table or JSON. The API keeps shared settings, storage, and the cached provider during its lifespan. Both call `analyze_image`, which controls validation, identification, nutrition lookup, total calculation, and persistence.

The service layer wraps calls that cross into `ai/` and adds retries, caching, logging, and error translation. The concurrency layer runs independent nutrition lookups in parallel. The storage layer is behind `HistoryRepository`; arrows into it cross the persistence boundary. Changing Anthropic to OpenAI normally requires only environment changes because provider creation remains behind the factory and wrapper.

2.2 OOP design & key abstractions

Required. Identify at least one example of inheritance or composition *in your own code* (not the provided package's). Quote 5–10 lines of the relevant class or object design. Justify why this abstraction was the right tool; if you used an ABC, explain why.

Defensibility: if a grader asked "why is this an abstract base class rather than a Protocol, a plain function with a callable parameter, or a configuration enum?" — what is your answer? Are there places where you used composition where inheritance would have been wrong, or vice versa?

Code listing:

```

1 class HistoryRepository(abc.ABC):
2     @abstractmethod
3     async def add(self, record: AnalysisRecord) -> None: ...
4
5     @abstractmethod
6     async def list_recent(self, limit: int = 20) -> list[AnalysisRecord]: ...
7
8 class PostgresHistoryRepository(HistoryRepository): ...
  
```

Justification (2–3 sentences):

The abstract class gives the memory and PostgreSQL repositories one asynchronous contract and lets the analyzer depend on behavior instead of database details. A configuration enum would only select a backend and would not define its operations. Composition was better for `CachingNutritionProvider`: it wraps any provider and adds caching and retries without copying provider code.

2.3 Data flow across module boundaries

Required. List the dataclasses you defined for your own SE layer. Confirm in one sentence that no naked dictionaries cross module boundaries.

The hardest call: when did you reuse a dataclass from the provided `ai/` package vs. wrap it in a new one for your SE layer? What was the trigger for wrapping?

The concurrency layer defines `LookupOutcome` and `PipelineResult`. `RetryPolicy` and the private cache `_Entry` are also dataclasses. The SE layer uses Pydantic models for `AnalysisResult`, `AnalysisRecord`, `AnalyzedIngredient`, and `MacroBreakdown`. We reuse the provided `Ingredient` and `NutritionFacts` while the data belongs to the AI stages. We wrap them when the application needs IDs, timestamps, warnings, storage paths, or per-ingredient status. No naked dictionaries cross the main module boundaries; dictionaries are created only for JSON and database conversion.

3. Concurrency & Performance

3.1 Why this concurrency model

Required. State which concurrency primitive you used (`asyncio`, `ProcessPoolExecutor`, `ThreadPoolExecutor`) and why. Identify the bottleneck the parallel design targets. State the bound (e.g. `Semaphore(10)`) and how you picked it.

Justify against alternatives: could you have used threads instead of `asyncio`? Why not? What is the smallest workload at which the concurrent design starts to win against sequential? What happens if you set the semaphore to 1 vs. 100?

Nutrition lookup is I/O-bound because each ingredient may require an independent HTTP request. The pipeline uses `asyncio.gather` and runs the blocking provider call with `asyncio.to_thread`. `Semaphore(10)` is the default bound, and configuration allows 1–100. A process pool would add unnecessary process cost because the bottleneck is network waiting, not CPU work. Direct threads could work, but `asyncio` fits FastAPI and asynchronous storage. With a bound of 1, the work becomes effectively sequential. A bound of 100 may trigger provider limits without useful extra speed. Ten is a conservative project default; production tuning needs live load tests and the provider's limits.

3.2 Sequential-vs-concurrent benchmark

Required. A table reporting wall-clock time for the same workload run sequentially vs. concurrently. State N (number of requests), the machine, the python version, whether caches were cleared, and the command line. Reproduce the table also in your README.

Honest interpretation: did you reach the theoretical $N \times$ speedup? If not, what was the new bottleneck (provider rate limit? GIL? network RTT? single-threaded DB writes?). What would push the curve further?

Workload	N	Sequential (s)	Concurrent (s)	Speedup
Simulated nutrition lookups	6	1.206	0.208	5.8×

Discussion (2–3 sentences):

The benchmark ran with Python 3.13.1 on the team's Windows computer using `python scripts/benchmark_pipeline.py`. It used six unique calls with 200 ms simulated latency and no external network or persistent cache. The result is close to 6× because most time is waiting; thread scheduling and pipeline overhead explain the remaining difference.

3.3 Rate limit & politeness

Required. State the rate-limit strategy (sleep-on-429, token bucket, leaky bucket, request budget). Document the configured budget. State the highest burst your test suite produces.

Failure-mode test: have you actually observed a 429 from the provider during development, or is this strategy untested in anger? What does the log look like when the budget is hit?

The application limits a batch to 10 concurrent nutrition calls by default. Provider errors are retried up to four times, with delays near 0.5, 1, and 2 seconds plus small jitter. A test sends 12 requests with a bound of 3 and confirms that no more than three are active. This is not a complete token bucket and does not yet read a `Retry-After` header. The team tested injected provider failures, not a live 429. Logs include the operation, attempt number, error, and delay. Production code should classify 429 and transient 5xx responses and follow the provider's requested waiting time.

4. Robustness & Error Handling**4.1 Wrapping the AI module**

Required. Describe your service-layer wrapper around `ai.*` calls: retries (which exceptions, how many attempts, backoff schedule), timeouts (per-call value), structured logging (what fields, at what level), input validation (what you check before calling, what you check on the response).

Pressure-test: what happens if the provider returns a 500? A 429? A connection reset mid-stream? A perfectly-formed JSON that contradicts the schema? Walk through one of these end-to-end.

`services/ai_service.py` wraps ingredient identification, logs the operation, retries `ProviderError`, and converts the final failure into `IngredientIdentificationError`. `CachingNutritionProvider` applies the same policy to nutrition calls and converts the last provider failure into `NutritionLookupError`. Defaults are four attempts, 0.5-second base delay, and an 8-second maximum delay.

Before the AI call, the program checks file existence, size, extension, declared type, and JPEG or PNG magic bytes. Pydantic validates structured outputs. INFO logs record stages and timings; WARNING records retries or failed ingredients; ERROR records final failures. Secrets are not logged. The current wrapper has no separate application-level timeout, which is a known gap that should be fixed with timeout-aware HTTP calls.

4.2 Failure-mode analysis (one concrete incident)

Required. Pick *one* failure mode you actually exercised (provider 5xx, timeout, malformed response, network drop, disk full, corrupted image, USDA outage, etc.). Describe: (i) how you injected the failure, (ii) what the system did, (iii) what the user saw, (iv) what the logs showed.

Go beyond "graceful degradation worked": what did you originally expect to happen vs. what actually happened? Did the test surface any design issue that you then fixed? Did the log give you enough to diagnose it without running the code again?

During a real API run, the VLM identified cooked broccoli, but USDA returned 403 **Forbidden**. The nutrition wrapper retried the provider call and converted the final error into a lookup failure. The concurrency pipeline kept successful ingredients instead of discarding the whole analysis.

The user received HTTP 200 with status **partial**, the affected item had status **lookup_failed**, and a warning explained that totals covered only the remaining ingredients. Logs showed the ingredient, attempts, retry delays, and final failure. We first expected a complete result; the incident showed that provider authentication must be checked separately. It also showed that 403 should fail fast rather than be treated like a temporary error.

4.3 Input validation

Required. For every entry point (CLI, HTTP, file, env), list the validation rules. State what error message the user gets when validation fails.

Adversarial: could a malformed image kill your service? A 100 MB JSON request? A query of 50,000 characters? A path that escapes your storage directory?

CLI: the path must exist and its content must pass size and magic-byte checks. Errors are printed clearly and return a non-zero exit code. **HTTP:** `/analyze` requires one multipart file; empty data, files above 5 MB, unsupported extensions, declared types, or signatures are rejected with structured JSON errors. **File handling:** the name is sanitized and limited to 60 characters, a UUID is added, and the temporary file is deleted in **finally**. **Environment:** Pydantic validates port, concurrency, cache TTL, image size, retry settings, and log level. **History:** `limit` is clamped to 1–200 and an unknown ID returns 404. A future version should reject oversized HTTP bodies while streaming instead of reading the full upload first.

5. Storage & Caching

5.1 Schema

Required. The SQLite schema for your domain tables. Include indices and foreign keys. State which fields are derived (cached) vs. source-of-truth.

Schema choices: why did you store embedding bytes as a blob rather than serialized JSON, or vice versa? What happens to the DB if the embedding dimension changes (a different provider, a different model)?

```
CREATE TABLE IF NOT EXISTS analysis_history (
  id          TEXT PRIMARY KEY,
  created_at  TIMESTAMPTZ NOT NULL,
  image_path  TEXT NOT NULL,
  image_filename TEXT NOT NULL,
  status      TEXT NOT NULL,
  ingredients JSONB NOT NULL,
  totals      JSONB NOT NULL,
  warnings    JSONB NOT NULL DEFAULT '[]',
);
CREATE INDEX IF NOT EXISTS analysis_history_created_at_idx
  ON analysis_history (created_at DESC);
```

5.2 Caching policy

Required. What do you cache, how long, where, and how is the cache invalidated. Report cache hit rate from a demo run (a single number is fine).

Correctness vs. freshness: which cached values can become stale (e.g. USDA nutrition for a renamed food, an LLM summary if the prompt changes)? How would you detect and evict them in production?

This project uses PostgreSQL, not the generic SQLite example in the template. It caches `NutritionFacts` in process by normalized ingredient name. The default TTL is 86,400 seconds (24 hours). Entries are removed on expiry, process restart, or an explicit `invalidate()` call. One repeated-name demo gives one miss followed by one hit, or a 50% hit rate across two lookups. A meal containing only new names has 0%. Values may become stale if USDA data or matching rules change; a production key should include provider and matching versions.

6. Testing

6.1 Strategy & coverage

Required. Report the coverage number (`pytest -cov`). State how you mocked the AI module and the HTTP layer. Confirm the provided `tests/test_ai_smoke.py` still passes.

Quality vs. coverage: which lines did you decide *not* to cover, and why? Which mocks did you have to redesign mid-project because they were testing the wrong thing?

Suite	Coverage
foodanalyzer/ (SE layer)	approximately 87%
Provided ai/ smoke tests	yes

6.2 Notable tests

Required. Highlight at least three tests: (i) one happy-path end-to-end test, (ii) one concurrency test (e.g. that `asyncio.gather` behaves correctly when one task raises), (iii) one error-path test that surfaces a real bug you would otherwise have missed.

What didn't you test? What is the test you wish you had time to write?

The suite is offline: fake VLM and nutrition providers replace live services, FastAPI is tested through its application interface, and storage tests use memory. The recorded run collected 97 tests; the command is `python -m pytest -q -cov=foodanalyzer -cov-report=term-missing`. The provided smoke tests still pass. Real PostgreSQL failures and live provider behavior are not fully covered because they require external services. Notable tests include: (i) an analyzer happy path that checks identified ingredients, totals, and persistence; (ii) `test_runs_in_parallel` and `test_semaphore_bounds_concurrency`; and (iii) `test_failures_are_captured_not_raised`, which proves one failed ingredient does not destroy successful results. Retry tests verify exact backoff without sleeping. The main missing test would start the API and PostgreSQL containers, write history, restart the API, and confirm persistence.

7. Deployment & Reproducibility

7.1 Docker image

Required. The exact `docker build` and `docker run` commands. The image size. The Python version baked in. Note whether you use a single-stage or multi-stage build.

Engineering trade-offs: did you ship an Alpine, slim, or full base image, and why? How does your image size compare to a fresh `python:3.12-slim` (~120 MB)?

The project uses a single-stage `python:3.12-slim` image. The measured local image size was about 416 MB. Slim avoids many Alpine binary-wheel problems, although the final image is larger than the base because it includes application dependencies.

```
docker build -t foodanalyzer .
docker run -rm -p 8000:8000 -env-file .env foodanalyzer
```

An offline CLI run is: `docker run -rm foodanalyzer python -m foodanalyzer analyze data/broccoli_egg.png -offline -no-store`.

7.2 Configuration

Required. A short list of every environment variable the app reads, and what happens if each is missing.

LLM_PROVIDER and LLM_MODEL select the VLM. One matching key is required for a real run: ANTHROPIC_API_KEY, OPENAI_API_KEY, or GOOGLE_API_KEY. NUTRITION_PROVIDER defaults to USDA and a real lookup needs USDA_API_KEY. Offline mode needs no keys. DATABASE_URL is optional; without a usable PostgreSQL URL, memory storage is used. Other variables have defaults: LOG_LEVEL=INFO, NUTRITION_CACHE_TTL_SECONDS=86400, MAX_IMAGE_SIZE_MB=5, MAX_CONCURRENCY=10, retry settings 4/0.5/8, HTTP_PORT=8000, and UPLOAD_DIR=uploads. Invalid numeric ranges or log levels stop settings validation.

8. Limitations & Future Work

Required. 3–5 bullet points. Honest list of where the system is brittle, what it would take to productionize, and what you would do differently with another week.

Be honest: no project is perfect. What was the worst design decision you made? What did you not get to test under load? What is the scenario most likely to break the system on a busy day?

- VLM ingredient names and weights can be wrong for hidden, mixed, or visually similar food. Results are estimates, not medical advice.
- Provider calls need explicit application-level timeouts and better error classification. A 403 should fail fast, while 429 and transient 5xx responses should use provider guidance.
- The cache belongs to one process. Multiple workers do not share it, and restart clears it; Redis would support shared caching.
- PostgreSQL needs automated container integration tests. Memory fallback is useful for demos but can hide configuration errors if warnings are ignored.
- The HTTP API reads the full upload before applying the custom size check. Public deployment should use streaming and server-level body limits.

9. Tools & Acknowledgements

Required. Disclose substantial AI-assistant use. For each significant LLM-aided contribution: which module, which assistant (Claude/ChatGPT/Cursor/Copilot/etc.), and whether the team reviewed and adapted it.

Cultural note: "AI helped with everything" is not a useful disclosure; "Cursor drafted the initial `retry_policy.py`; we rewrote backoff logic after observing it hit the rate limit" is.

The project uses Python, FastAPI, Pydantic, pydantic-settings, asyncpg, pytest, pytest-asyncio, pytest-cov, httpx, Docker, a configurable VLM, and USDA FoodData Central. Git and GitHub were used for branches and pull requests. Claude drafted the initial implementation of the retry/cache/concurrency services, the orchestrator, the CLI, and the FastAPI service under `foodanalyzer/` based on our view of the project. Other modules such as configuration, the exception hierarchy, logging, upload validation, both storage backends were adapted to the given concept and integrated with it. ChatGPT was used to explain code, propose ideas for the test cases, and draft this report from the supplied template, project brief. Later the report was simplified and adapted to the project.

References

- [1] FastAPI documentation: <https://fastapi.tiangolo.com/>
- [2] Pydantic documentation: <https://docs.pydantic.dev/>
- [3] Python `asyncio` documentation: <https://docs.python.org/3/library/asyncio.html>
- [4] USDA FoodData Central API Guide: <https://fdc.nal.usda.gov/api-guide>

A. Appendix — Reproducing the benchmark

Exact commands to reproduce the §3 benchmark. Anyone with the repo and a valid `.env` should be able to run the lines below and get numbers within 20% of the ones in the table.

```
$ git clone https://github.com/atlukhanovmurad2-ui/swe_project.git
$ cd swe_project
$ python -m venv .venv
$ pip install -r requirements.txt
$ python scripts/benchmark_pipeline.py
```

End of report — thank you for reading.